Un emulatore dell'Intel 8086 verificato sull'hardware reale

Leonardo Marzioni
Ingegneria Informatica e dell'Automazione
Università Politecnica delle Marche, Ancona
Matricola S1110037

***Sommario*—Questo lavoro presenta un emulatore di un sottoinsieme dell'Intel 8086, scritto in C++20, con un debugger da terminale. Il modello del processore è ricavato dal manuale Intel della famiglia 8086, mentre la correttezza è verificata sull'hardware reale con la suite SingleStepTests, registrata su un 8088. Le regole dell'emulatore che la suite può verificare, dalle formule dei flag al calcolo degli indirizzi, sono state confrontate con i suoi test prima di scriverne il codice, e gli strumenti principali di verifica sono stati provati anche su dati sbagliati di proposito. Su 1.340.000 test della suite nessuno fallisce; i casi che la suite non raggiunge sono coperti da test unitari. Il lavoro mostra anche dove il manuale, pur essendo la specifica, non basta: non dice nulla delle codifiche che dichiara non usate e in alcuni punti descrive i passi di un'istruzione in un ordine diverso da quello dell'hardware.**

I. Introduzione

Un emulatore è un programma che riproduce il comportamento di un processore: legge le istruzioni da una memoria simulata e ne applica gli effetti a registri, flag e memoria. Questo lavoro presenta *i8086emu*, un emulatore di un sottoinsieme dell'Intel 8086 con un debugger da terminale. Il sottoinsieme, definito nella proposta di progetto, comprende le istruzioni di trasferimento (MOV), aritmetiche e logiche (ADD, SUB, CMP, AND, OR, XOR, INC, DEC, NEG, NOT), di stack (PUSH, POP) e di salto e chiamata (JMP, JZ, JNZ, CALL, RET), più NOP e HLT, con tutti i modi di indirizzamento a 16 bit e i prefissi di segmento.

Il riferimento naturale è il manuale Intel [1]. Un emulatore scritto solo a partire dal manuale, però, ne eredita le lacune: come si vedrà, il manuale non dice nulla delle codifiche che dichiara non usate, nelle pagine consultate non tratta alcuni casi estremi come il superamento dell'ultimo indirizzo fisico, e in qualche punto descrive i passi di un'istruzione in un ordine diverso da quello del processore. Per questo l'emulatore è stato verificato anche sull'hardware reale, con SingleStepTests [2]: una suite di test registrati su un processore fisico, che per ogni istruzione riporta lo stato prima e dopo l'esecuzione.

I contributi del lavoro sono:

- un emulatore in C++20 con decodifica guidata da tabelle, e un debugger che si può provare senza terminale (Sezione III);
- un harness che esegue 1.340.000 test della suite, nessuno dei quali fallisce (Sezioni VI e VII);
- un metodo di verifica in cui le regole che la suite può verificare sono confrontate con i suoi test prima di scriverne il codice, e i controlli principali sono provati anche su dati sbagliati (Sezione VI);
- un elenco dei casi in cui manuale e hardware non coincidono, con la scelta adottata per ciascuno (Sezioni IV e V).

La Sezione II descrive il modello dell'8086 secondo il manuale e la Sezione III l'architettura dell'emulatore. Le Sezioni IV e V trattano i casi limite e i disallineamenti tra documentazione e hardware, la Sezione VI il metodo di verifica e i test, la Sezione VII i risultati. La Sezione VIII discute limiti e sviluppi, e la Sezione IX conclude.

II. Il modello dell'8086

Questa sezione riassume il modello del processore che l'emulatore riproduce. La fonte è il manuale Intel della famiglia 8086 [1]: salvo indicazione diversa, le pagine, le figure e le tabelle citate tra parentesi sono quelle del manuale. Dove serve il comportamento dell'hardware reale si cita la suite SingleStepTests [2], descritta nella Sezione VI-C.

A. Panoramica

L'8086 e l'8088 sono due processori a 16 bit strettamente imparentati. L'8088 ha un percorso dati esterno a 8 bit invece che a 16; per il resto i due processori sono identici, e il software scritto per uno gira sull'altro senza modifiche (p. 2-1).

Il manuale descrive l'esecuzione di un programma come la ripetizione di quattro passi: lettura dell'istruzione dalla memoria, lettura di un eventuale operando, esecuzione, scrittura del risultato (pp. 2-3, 2-4). Nell'8086 questi passi sono divisi tra due unità che lavorano in parallelo (p. 2-5):

- l'*Execution Unit* (EU) esegue le istruzioni. Contiene l'ALU a 16 bit, i registri generali e i flag, e non ha alcun collegamento con il bus. Le EU dei due processori sono identiche;
- la *Bus Interface Unit* (BIU) esegue tutti gli accessi al bus. Contiene i registri di segmento e l'instruction pointer, calcola gli indirizzi fisici e, mentre l'EU è occupata, legge in anticipo le istruzioni successive in una coda (*prefetch queue*) di 6 byte sull'8086 e di 4 sull'8088 (pp. 2-5, 2-6). Quando l'EU esegue un salto, la BIU svuota la coda e riprende a leggere dal nuovo indirizzo (p. 2-6).

Progetto per il corso di Architetture dei Calcolatori e Cloud Computing, A.A. 2026/2027.

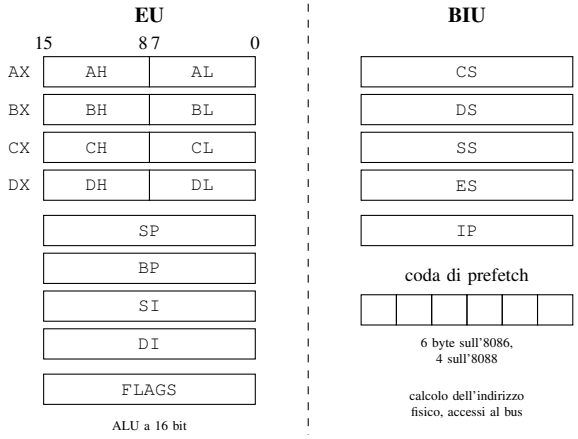


Figura 1. I registri dell'8086 divisi tra le due unità (dalle Fig. 2-6, 2-7 e 2-8 del manuale). A sinistra i registri dati con le metà a 8 bit, i puntatori e indici e i flag; a destra i registri di segmento, IP e la coda, che l'emulatore non modella.

La sovrapposizione tra lettura ed esecuzione serve alle prestazioni: il programmatore vede comunque una sequenza di istruzioni eseguite una alla volta. L'emulatore modella proprio questo livello. Dell'EU riproduce la semantica di ogni istruzione, della BIU soltanto il calcolo degli indirizzi; la coda e i tempi dei cicli di bus non sono modellati. La Fig. 1 mostra i registri divisi tra le due unità.

B. Registri

I registri generali sono otto, tutti a 16 bit, divisi in due gruppi (p. 2-6):

- i *registri dati* AX, BX, CX e DX, le cui metà alta e bassa si possono usare anche come registri a 8 bit separati (AH e AL, BH e BL, e così via);
- i *puntatori e indici* SP, BP, SI e DI, accessibili solo a 16 bit.

Alcune istruzioni usano certi registri in modo implicito (Tabella 2-1); tra questi usi, nel sottoinsieme emulato compare solo quello di SP nelle operazioni sullo stack. Le forme con l'accumulatore (AL o AX) del sottoinsieme sono invece codifiche più corte di operazioni che esistono anche nella forma generale.

I registri di segmento CS, DS, SS ed ES contengono la base dei quattro segmenti indirizzabili in ogni momento: codice, dati, stack ed extra (p. 2-7). L'instruction pointer IP corrisponde al program counter dell'8080. Il manuale precisa che IP contiene l'offset, nel segmento di codice, della prossima istruzione da *leggere*; quando viene salvato sullo stack, però, è prima corretto per puntare alla prossima istruzione da *eseguire* (p. 2-7). Senza coda le due cose coincidono: nell'emulatore IP indica sempre la prossima istruzione da eseguire, ed è questa la base da cui partono i salti relativi e l'indirizzo di ritorno di CALL (Sezione III-G). I programmi non accedono a IP direttamente, ma solo attraverso le istruzioni che lo modificano (p. 2-7).

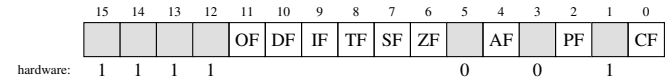


Figura 2. La parola dei flag (dalla Fig. 2-32 del manuale). In grigio i bit riservati, che per il manuale hanno valore indeterminato; sotto, il valore che hanno sempre nella suite.

C. Il registro dei flag

I flag sono nove: sei flag di stato, che descrivono il risultato di un'operazione aritmetica o logica, e tre flag di controllo, che modificano il funzionamento del processore (pp. 2-7, 2-8). Il manuale ne dà la posizione nella parola a 16 bit salvata e ripristinata da PUSHF e POPF (Fig. 2-32, p. 2-33), riportata nella Fig. 2.

Le regole generali dei flag di stato sono (p. 2-35):

- **CF**: riporto dal bit più significativo del risultato in un'addizione, prestito verso di esso in una sottrazione;
- **AF**: riporto o prestito tra il nibble basso e il nibble alto; serve alle istruzioni di aggiustamento decimale;
- **SF**: copia del bit più significativo del risultato (bit 7 o 15);
- **ZF**: risultato uguale a zero;
- **PF**: numero pari di bit a 1 negli *otto bit bassi* del risultato, anche nelle operazioni a 16 bit;
- **OF**: overflow con segno, cioè un risultato che non sta nella destinazione come numero in complemento a due.

Alcune istruzioni fanno eccezione. INC e DEC non modificano CF (pp. 2-35, 2-36). NEG mette CF a 1 tranne quando l'operando è zero, e negare -128 o -32768 lascia l'operando invariato e attiva OF (p. 2-36). CMP calcola la stessa sottrazione di SUB senza scriverne il risultato (p. 2-36). AND, OR e XOR azzerano OF e CF, aggiornano SF, ZF e PF dal risultato e lasciano AF indefinito; NOT non modifica alcun flag (p. 2-38).

I flag di controllo (DF per le istruzioni stringa, IF per le interruzioni mascherabili, TF per l'esecuzione passo passo, p. 2-8) non sono usati da nessuna istruzione del sottoinsieme.

Indefinito non vuol dire casuale: l'hardware produce comunque un valore, e la suite lo registra. Per confrontare solo ciò che il manuale definisce, la suite fornisce per ogni istruzione una maschera dei flag indefiniti [2]. Un caso analogo sono i bit riservati della parola dei flag, che per il manuale hanno un valore indeterminato (Fig. 2-32). Abbiamo controllato tutti i valori di FLAGS presenti nei 134 file della suite usati dal progetto, stati iniziali e finali, 2.114.663 in tutto: i bit 1 e 12–15 valgono sempre 1, i bit 3 e 5 sempre 0. L'emulatore adotta questo comportamento (Sezione III-D).

D. Memoria segmentata

a) Organizzazione: La memoria è un array di 1.048.576 byte (p. 2-8). Istruzioni, byte e parole possono stare a qualsiasi indirizzo, senza vincoli di allineamento; una parola ha il byte meno significativo all'indirizzo più basso (pp. 2-8, 2-10).

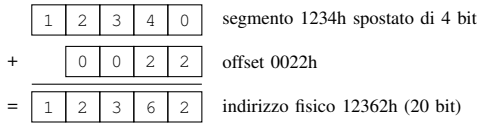


Figura 3. Calcolo dell'indirizzo fisico dall'indirizzo logico 1234h:0022h, con l'esempio della Fig. 2-18 del manuale. Ogni casella è una cifra esadecimale.

Tabella I
SEGMENTO PREDEFINITO PER TIPO DI RIFERIMENTO (DALLA TABELLA 2-2)

Riferimento	Predefinito	Con prefisso	Offset
Lettura di istruzioni	CS	–	IP
Operazioni sullo stack	SS	–	SP
Dato in memoria	DS	CS, ES, SS	EA
Dato con base BP	SS	CS, DS, ES	EA

EA è l'indirizzo effettivo (Sezione II-E). Le righe delle istruzioni stringa sono omesse perché fuori dal sottoinsieme.

b) Segmenti e indirizzo fisico: Un programma vede la memoria come un insieme di *segmenti*: blocchi contigui di al più 64 KB che iniziano a un multiplo di 16 byte e possono essere adiacenti, separati o sovrapposti in parte o del tutto (p. 2-10). Ogni locazione ha quindi due indirizzi (pp. 2-11, 2-12). L'*indirizzo logico* è una coppia segmento:offset di due valori a 16 bit senza segno. L'*indirizzo fisico*, a 20 bit, va da 0 a FFFFFh: la BIU lo calcola spostando il segmento di quattro bit a sinistra e sommando l'offset (Fig. 3). Più indirizzi logici possono indicare la stessa locazione fisica.

Questo calcolo ha due conseguenze ai bordi. Gli offset sono aritmetica modulo 64 KB: dalla fine di un segmento si torna all'inizio dello *stesso* segmento (p. 2-12); la Sezione IV-B ne mostra l'effetto su una parola all'offset FFFFh. Una somma che supera FFFFFh, invece, nelle pagine consultate del manuale non è trattata: il README della suite precisa che sull'8088 lo spazio di indirizzamento ricomincia da zero dopo FFFFFh [2], e l'emulatore fa lo stesso.

c) Segmento predefinito: Ogni accesso usa un segmento predefinito che dipende dal tipo di riferimento (Tabella I). Le istruzioni si leggono sempre da CS:IP e lo stack è sempre SS:SP; per i dati il segmento predefinito è DS, oppure SS quando BP fa da registro base. Un prefisso di un byte, il *segment override*, sostituisce il segmento predefinito di un operando in memoria (p. 2-13).

d) Stack: Lo stack è largo 16 bit e sta nel segmento indicato da SS; SP contiene l'offset della cima. PUSH decrementa SP di 2 e poi scrive la parola, POP la legge e poi incrementa SP di 2: lo stack cresce verso gli indirizzi bassi (p. 2-14). Uno stack che supera i 64 KB sovrascrive l'inizio del proprio segmento (p. 2-14), ancora una volta per l'aritmetica modulo 64 KB.

e) Zone riservate: Le locazioni da 0 a 7Fh e da FFFF0h a FFFFFh sono dedicate alle interruzioni e al reset (pp. 2-14, 2-15). Nell'emulatore non sono usate, perché interruzioni e ROM di avvio sono fuori scope.

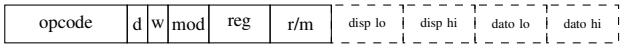


Figura 4. Formato tipico di un'istruzione (dalla Fig. 4-20 del manuale). Il ModR/M c'è solo nelle istruzioni con un operando r/m (registro o memoria); spiazzamento e dato immediato, se presenti, sono di uno o due byte, con il byte meno significativo per primo.

E. Codifica delle istruzioni

Un'istruzione occupa da 1 a 6 byte (p. 4-18), e la maggior parte segue il formato della Fig. 4 (Fig. 4-20, p. 4-19). Nel primo byte i sei bit alti sono l'opcode; nelle istruzioni con due operandi seguono il bit D, che indica se il campo REG è la destinazione (1) o la sorgente (0), e il bit W, che sceglie tra byte (0) e parola (1). Nelle istruzioni con un immediato il bit S chiede di estendere con segno a 16 bit un immediato di 8 bit (Tabella 4-7).

Il secondo byte, detto ModR/M, ha tre campi (p. 4-20):

- **MOD** (Tabella 4-8): 11 indica che entrambi gli operandi sono registri; 00, 01 e 10 indicano un operando in memoria seguito da nessuno, uno o due byte di spiazzamento;
- **REG** (Tabella 4-9): un registro, nell'ordine AL, CL, DL, BL, AH, CH, DH, BH a 8 bit e AX, CX, DX, BX, SP, BP, SI, DI a 16 bit. In alcune istruzioni, soprattutto quelle con un immediato e un operando in memoria, REG estende invece l'opcode: un solo opcode, detto *di gruppo*, codifica fino a otto operazioni diverse;
- **R/M** (Tabella 4-10): con MOD = 11 è il secondo registro, altrimenti sceglie come calcolare l'indirizzo dell'operando.

Seguono lo spiazzamento e l'immediato, in quest'ordine: gli immediati seguono sempre gli spiazzamenti. Uno spiazzamento di un solo byte viene esteso con segno a 16 bit (p. 4-20).

a) Indirizzo effettivo: L'offset di un operando in memoria si chiama *indirizzo effettivo* (EA): è un numero a 16 bit senza segno che l'EU calcola sommando un registro base (BX o BP), un registro indice (SI o DI) e uno spiazzamento, in qualsiasi combinazione (p. 2-68, Fig. 2-34). Ne derivano i modi di indirizzamento diretto, indiretto tramite registro, con base, con indice e con base e indice (pp. 2-69–2-71). La Tabella II riporta le combinazioni della Tabella 4-10. C'è un'eccezione: con MOD = 00 e R/M = 110 l'operando non è [BP] ma un indirizzo diretto a 16 bit. Per questo [BP] senza spiazzamento si codifica con MOD = 01 e uno spiazzamento nullo.

b) Prefissi: I registri di segmento hanno un codice di due bit: 00 per ES, 01 per CS, 10 per SS, 11 per DS (Tabella 4-11). Il prefisso di segment override ha la forma $001\,sr\,110$, cioè 26h, 2Eh, 36h e 3Eh (Tabella 4-12), e lo stesso codice compare nel campo REG di MOV da e verso un registro di segmento.

F. Il sottoinsieme emulato

La Tabella III elenca le istruzioni emulate, con gli opcode e i flag modificati. Tutti i modi di indirizzamento e i prefissi

Tabella II
INDIRIZZO EFFETTIVO PER R/M (DALLA TABELLA 4-10)

R/M	EA con MOD = 00	Segmento predefinito
000	BX + SI	DS
001	BX + DI	DS
010	BP + SI	SS
011	BP + DI	SS
100	SI	DS
101	DI	DS
110	spiazzamento a 16 bit (diretto)	DS
111	BX	DS

Con MOD = 01 si somma uno spiazzamento a 8 bit esteso con segno, con MOD = 10 uno a 16 bit; con R/M = 110 la base diventa BP, e il segmento predefinito SS. La somma si tronca a 16 bit.

Tabella III
IL SOTTOINSIEME EMULATO

Istruzione	Opcode (hex)	Flag modificati
MOV	88–8B, 8C, 8E, A0–A3, B0–BF, C6 e C7 /0	–
ADD, SUB, CMP	00–05, 28–2D, 38–3D; 80–83 /0, /5, /7	tutti i flag di stato
AND, OR, XOR	20–25, 08–0D, 30–35; 80, 81, 83 /4, /1, /6	OF = CF = 0; SF, ZF, PF; AF indefinito
INC, DEC	40–4F; FE e FF /0, /1	tutti tranne CF
NEG	F6 e F7 /3	tutti i flag di stato
NOT	F6 e F7 /2	–
PUSH, POP	50–57, 58–5F	–
JMP	EB, E9, FF /4	–
JZ, JNZ	74, 75	–
CALL, RET	E8, FF /2; C3	–
NOP, HLT	90, F4	–

La notazione /*n* indica il valore del campo REG di un opcode di gruppo. Solo forme *near*: JMP, CALL e RET restano nel segmento di codice. Le forme 82 e 83 con REG = 1, 4, 6 sono discusse nella Sezione V.

di segmento sono supportati.

La codifica delle operazioni aritmetiche e logiche ha una regolarità che l'emulatore sfrutta (Tabella 4-12). Negli opcode da 00h a 3Fh i bit 5–3 scelgono l'operazione, nell'ordine ADD, OR, ADC, SBB, AND, SUB, XOR, CMP, e i bit 2–0 la forma degli operandi: r/m e registro a 8 o 16 bit nei due versi (qui il bit 1 è D e il bit 0 è W), oppure accumulatore e immediato. Così ADD occupa 00h–05h, OR 08h–0Dh, e via di seguito. Il campo REG degli opcode di gruppo 80h–83h, che hanno un operando r/m e un immediato, usa lo stesso ordine (Tabella 4-13). ADC e SBB non fanno parte del sottoinsieme.

G. Controllo del processore

Dopo un RESET i flag sono azzerati, CS vale FFFFh, IP, DS, SS ed ES valgono 0 e la coda è vuota (Tabella 2-4, p. 2-29). La prima istruzione eseguita è quindi a FFFF0h, dove di norma si trova un salto all'inizio del programma di sistema (p. 2-29). L'emulatore non ha una ROM di avvio e non riproduce questo reset: azzera i registri e lascia che lo stato iniziale di un programma lo prepari il caricatore (Sezione III-I). I bit riservati di FLAGS non sono flag, quindi "flag azzerati" è compatibile con il valore F002h della parola (Sezione II-C).

HLT porta il processore nello stato di halt, da cui si esce con un RESET, con un'interruzione non mascherabile o, se le interruzioni sono abilitate, con una mascherabile; HLT non modifica i flag (pp. 2-29, 2-48). Le interruzioni sono fuori dal sottoinsieme, quindi nell'emulatore l'halt dura fino al reset; dove punti IP dopo HLT è discusso nella Sezione IV-C.

H. 8086 e 8088

SingleStepTests è stata registrata su un processore fisico, un AMD D8088 (8441DMA) del 1982, collegato all'interfaccia Arduino8088 [2]. L'emulatore invece modella un 8086. La suite è comunque un oracolo valido, per tre motivi:

- l'EU, che determina il risultato delle istruzioni, è la stessa nei due processori (p. 2-5), e il software gira sull'uno o sull'altro senza modifiche (p. 2-1);
- le differenze stanno tutte nella BIU: larghezza del bus dati, lunghezza della coda (4 byte contro 6) e momento in cui la BIU la riempie (p. 2-6), accessi alla memoria per byte invece che per parola (p. 2-15). Il manuale le definisce invisibili ai programmi (pp. 2-8, 2-15);
- nella suite questi aspetti stanno in campi separati, la coda (`queue`) e i cicli di bus (`cycles`), che l'harness ignora (Sezione VI-C).

Il problema sono piuttosto le forme che il manuale dichiara non usate: sull'hardware hanno comunque un comportamento preciso, che la suite misura. Come vengono trattate è spiegato nella Sezione V.

III. ARCHITETTURA DELL'EMULATORE

Questa sezione descrive come il modello della Sezione II è stato tradotto in codice. Per ogni scelta si indicano anche le alternative scartate.

A. Ambiente e strumenti

L'emulatore è scritto in C++20 e sviluppato su WSL 2 con Ubuntu 24.04, GCC 13 e CMake con Ninja. Dello standard C++20 si usano `std::popcount` per il flag di parità, `std::span`, l'`operator==` generato dal compilatore, `using enum`, `constinit` per le tabelle costruite a tempo di compilazione e `std::format` per l'output esadecimale del debugger.

Il codice compila senza avvisi con le opzioni abituali di GCC, cioè `-Wall`, `-Wextra` e `-Wpedantic`, e con tre opzioni più severe: `-Wshadow`, `-Wconversion` e `-Wsign-conversion`. Le ultime due sono particolarmente utili in un emulatore: nelle espressioni C++ i tipi `uint8_t` e `uint16_t` vengono promossi a `int`, e il risultato va riportato a 8 o 16 bit, che è proprio la semantica dell'8086. Il compilatore segnala i troncamenti impliciti che nascono dal mescolare larghezze diverse, come una parola copiata in un byte o una somma con un `int`; quando gli operandi hanno già la larghezza della destinazione, come in `~x` o `a - b`, non avvisa, e il codice usa comunque un cast esplicito per convenzione. Le build di debug usano AddressSanitizer, UndefinedBehaviorSanitizer e `_GLIBCXX_ASSERTIONS`, che controlla i limiti di `std::array` e `std::vector`.

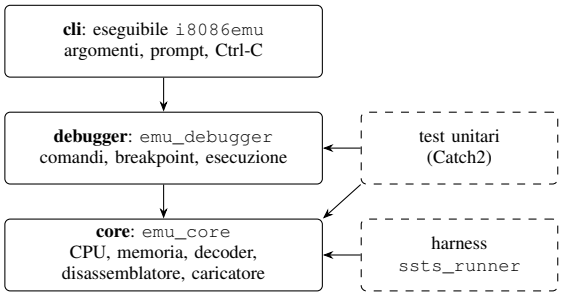


Figura 5. I tre livelli del codice. Le frecce indicano chi usa chi; il core non fa I/O e non ha dipendenze.

Due preset CMake, `debug` e `release`, eseguono configurazione, compilazione e test con un solo comando. Le due dipendenze, Catch2 [3] per i test unitari e nlohmann/json [4] per leggere la suite, sono scaricate da CMake e verificate con il loro hash SHA-256: un tag git si può spostare, un hash no.

B. Tre livelli

Il codice è diviso in tre livelli (Fig. 5). Il *core* contiene CPU, memoria, decoder, disassemblatore e caricatore; non fa I/O e non ha dipendenze, quindi si può provare da solo. Il *debugger* contiene i comandi e la logica di esecuzione, ma non conosce il terminale: riceve una riga di comando e scrive il risultato su uno stream. La *CLI* aggiunge soltanto ciò che riguarda il terminale: argomenti, prompt e Ctrl-C. L'harness di SingleStepTests usa direttamente il core, i test unitari anche il debugger.

C. Memoria

La memoria è un vettore di 1 MB allocato sullo heap. Un array della stessa dimensione dentro l'oggetto finirebbe sullo stack ogni volta che una `Memory` è una variabile locale, come nei test, e su Windows lo stack predefinito è proprio di 1 MB.

Gli accessi sono di due tipi:

- quelli per segmento:offset, `read8`, `read16`, `write8` e `write16`, usati dalla CPU e dai comandi del debugger. L'indirizzo fisico è $((seg \ll 4) + off)$ & FFFFFh, calcolato a 32 bit, con il wrap a 1 MB della Sezione II-D. Il byte alto di una parola sta all'offset successivo, con wrap a 16 bit dentro lo stesso segmento;
- quelli per indirizzo fisico, `peek` e `poke`, esterni al bus della CPU. Li usano il caricatore, l'harness, perché la suite descrive la memoria per indirizzo fisico, e i test.

Un'interfaccia `Bus` astratta, con metodi virtuali, è stata scartata: servirebbe per i dispositivi mappati in memoria, ma l'I/O è fuori dal sottoinsieme.

D. Registri e flag

Gli enum `Reg16`, `Reg8` e `SegReg` elencano i registri nell'ordine della loro codifica nel ModR/M (Tabelle 4-9 e 4-11 del manuale), quindi il decoder converte un campo di 3 bit con un semplice cast. La codifica dei registri a 8 bit ha una struttura utile: nell'indice di `Reg8` il bit 2 sceglie la metà (0 bassa, 1 alta) e i bit 1–0 il registro a 16 bit, per cui AH = 4 è la metà alta di AX = 0. Le metà si leggono e si scrivono con shift e maschere sulla parola. Una `union` tra una parola e due byte sarebbe stata più corta, ma in C++ leggere un membro diverso dall'ultimo scritto è comportamento indefinito, e il risultato dipenderebbe anche dall'endianness della macchina che esegue l'emulatore.

`Registers` è un tipo valore con l'uguaglianza generata dal compilatore: test e harness confrontano stati interi con un solo ==. La CPU riceve invece un riferimento alla memoria e non la possiede, perché CPU e memoria sono componenti distinti collegati dal bus e il debugger accede alla memoria anche senza passare dalla CPU. Per lo stesso motivo una `Cpu` non si può copiare.

FLAGS è conservato come parola a 16 bit con il layout della Fig. 2, non come una struttura di booleani: debugger e harness lavorano direttamente sulla parola. All'interno sono memorizzati solo i nove bit definiti (maschera 0FD5h); in lettura i bit riservati assumono il valore F002h osservato sull'hardware. Così, dopo l'azzeramento dei registri, che non è il reset hardware della Sezione II-G, tutti i flag sono a zero e la parola FLAGS vale F002h.

È stata scartata la tecnica dei *lazy flags*: memorizzare operandi e tipo dell'ultima operazione e calcolare i flag solo quando un'istruzione li legge. Fa risparmiare tempo, ma qui la velocità non è un requisito, e il codice sarebbe più difficile da verificare e da spiegare.

E. ALU

Le operazioni aritmetiche e logiche sono funzioni libere, senza stato: ricevono gli operandi, la larghezza e un riferimento ai flag, aggiornano i flag e restituiscono il risultato. Si provano quindi senza memoria né decoder. L'enum delle operazioni segue l'ordine dei bit 5–3 degli opcode (Sezione II-F); ADC e SBB vi compaiono solo per non rompere la numerazione. La larghezza (byte o parola) è un parametro a runtime e non un parametro template: è un dato dell'istruzione decodificata, il bit W, e un template costringerebbe a scegliere tra due istanze in ogni punto dell'esecutore.

Siano a e b gli operandi, r il risultato troncato alla larghezza, M la maschera (FFh o FFFFh) e S il bit di segno (80h o 8000h). Per l'addizione e la sottrazione:

$$\mathrm{CF}_{\mathrm{ADD}} = (a + b > M), \qquad \mathrm{CF}_{\mathrm{SUB}} = (a < b), \tag{1}$$

$$\mathrm{OF}_{\mathrm{ADD}} = (a \oplus r)\ \&\ (b \oplus r)\ \&\ S, \tag{2}$$

$$\mathrm{OF}_{\mathrm{SUB}} = (a \oplus b)\ \&\ (a \oplus r)\ \&\ S, \tag{3}$$

$$\mathrm{AF} = (a \oplus b \oplus r)\ \&\ 10\mathrm{h}. \tag{4}$$

In un'addizione c'è overflow quando gli operandi hanno lo stesso segno e il risultato ha segno diverso da entrambi; in una sottrazione quando gli operandi hanno segno diverso e il risultato ha segno diverso dal minuendo. Il bit 4 di $a \oplus b \oplus r$ vale 1 esattamente quando nel bit 4 entra un riporto o ne esce un prestito, cioè quando AF deve valere 1. ZF, SF e PF dipendono solo da r; PF si calcola con `std::popcount` sul solo byte basso.

```
// bit 5..3: operazione (ordine del gruppo 1)
// bit 2..0: forma degli operandi
for (std::size_t op = 0; op < 8; ++op) {
    const Mnemonic m = kGroup1[op];
    if (m == Unimplemented) continue; // ADC, SBB
    const std::size_t base = op << 3;
    t[base + 0] = {m, Eb, Gb};
    t[base + 1] = {m, Ev, Gv};
    t[base + 2] = {m, Gb, Eb};
    t[base + 3] = {m, Gv, Ev};
    t[base + 4] = {m, AL, Ib};
    t[base + 5] = {m, AX, Iv};
}
```

Figura 6. Generazione del blocco ALU 00h–3Fh della tabella degli opcode (da `opcode_table.cpp`, commenti abbreviati).

Le altre operazioni riusano queste due. INC e DEC sono un'addizione e una sottrazione con 1 che non aggiornano CF. NEG è la sottrazione $0 - a$: il caso particolare del manuale, CF a 1 tranne quando l'operando è zero, è semplicemente il prestito di $0 - a$, e anche OF con l'operando -128 o -32768 esce dalla formula (3). AND, OR e XOR azzerano CF e OF e aggiornano ZF, SF e PF; AF, indefinito, viene azzerato. NOT non riceve nemmeno i flag. Prima di scrivere il codice, queste formule sono state confrontate con la suite (Sezione VI-B).

F. Decodifica guidata da tabelle

La decodifica parte da una tabella di 256 voci indicizzata dal byte di opcode. Ogni voce contiene il mnemonico e il tipo dei due operandi, in una notazione compatta ispirata a quella delle mappe degli opcode di Intel: `Eb` ed `Ev` per un operando r/m a 8 o 16 bit, `Gb` e `Gv` per il registro del campo REG, `Ib` e `Iv` per un immediato, `Ibs` per l'immediato esteso con segno di 83h, `Jb` e `Jv` per lo spiazzamento di un salto, e così via.

La tabella è costruita a tempo di compilazione da una funzione `constexpr` e inizializzata con `constinit`. I blocchi regolari della codifica sono generati da cicli, che rendono visibile la regolarità della Sezione II-F invece di elencare le voci a mano (Fig. 6). Gli opcode di gruppo rimandano a sotto-tabelle di otto mnemonici indicizzate dal campo REG; 82h ne ha una propria, con le sole operazioni documentate (Sezione V).

La funzione `decode()` legge i byte nell'ordine del manuale: gli eventuali prefissi di segmento, l'opcode, il ModR/M, lo spiazzamento e infine gli immediati; come tratta più prefissi è spiegato nella Sezione IV-B. Il risultato è una struttura `Instruction` con il mnemonico, la larghezza, la lunghezza totale e gli operandi già risolti. Due scelte la rendono riusabile:

- un operando in memoria resta *simbolico*: base, indice, spiazzamento e segmento, non l'indirizzo calcolato, che dipende dai registri al momento dell'esecuzione. Il disassemblatore può così scrivere `[bp+si-0x2]` invece di un numero;
- per i salti relativi il decoder calcola subito la destinazione assoluta, sommando lo spiazzamento all'offset dell'istruzione successiva.

Soprattutto, `decode()` è una funzione pura: legge la memoria ma non modifica nulla. Il disassemblatore la riusa così com'è, e un opcode non implementato lascia la CPU esattamente com'era. È stato scartato lo stile con un gestore per opcode che decodifica ed esegue da sé: è veloce, ma avrebbe duplicato la decodifica nel disassemblatore.

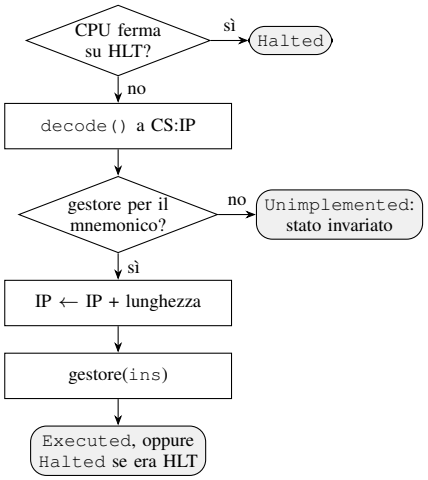


Figura 7. Esecuzione di una istruzione in `Cpu::step()`. IP avanza prima del gestore, così salti relativi e CALL partono dall'istruzione successiva.

G. Il ciclo di esecuzione

`Cpu::step()` esegue una sola istruzione, prefissi compresi: la stessa granularità dei test della suite e del comando `step` del debugger. Il flusso è quello della Fig. 7. Se la CPU è ferma su HLT non esegue nulla. Altrimenti decodifica l'istruzione a CS:IP e cerca il gestore del suo mnemonico in una tabella di puntatori a funzione membro. Se il gestore non c'è, restituisce `Unimplemented` con l'opcode e, per gli opcode di gruppo, il campo REG (ad esempio `F6 /4`); registri e memoria sono intatti e IP indica ancora il primo prefisso. Altrimenti porta IP all'istruzione successiva e solo dopo chiama il gestore, come segue dalla regola di p. 2-7 del manuale (Sezione II-B): i salti relativi e l'indirizzo di ritorno di CALL partono da lì.

I gestori sono uno per mnemonico, e le sei operazioni ALU ne condividono uno solo, un template sull'operazione che per CMP, con `if constexpr`, non scrive il risultato. Gli operandi si leggono e si scrivono attraverso due funzioni generiche, che distinguono registro a 8 o 16 bit, registro di segmento, immediato e memoria; le parole dello stack passano invece per SS:SP con due funzioni dedicate. Per un operando in memoria l'indirizzo effettivo si calcola qui, come somma a 32 bit troncata a 16. Grazie a questa struttura un gestore è di poche righe: MOV, ad esempio, è la scrittura nella destinazione del valore letto dalla sorgente. L'ordine in cui un gestore legge e scrive basta a riprodurre casi come `PUSH SP` e `CALL SP` (Sezione IV-A).

H. Disassemblatore

Il disassemblatore sta nel core e restituisce stringhe. Chiama `decode()` e formatta l'`Instruction`, quindi decodifica ed esecuzione non possono divergere. La sintassi è quella di NASM [5] con le convenzioni del suo disassemblatore,

ndisasm, che fa da oracolo nei test (Sezione VI-A): minuscole, esadecimale `0x..` senza zeri iniziali, `byte` o `word` solo quando nessun registro rende evidente la dimensione (`mov byte [bx],0x42`), spiazzamenti con segno (`[bp-0x2]`), prefisso dentro le parentesi (`[es:bx]`) e destinazioni dei salti come offset assoluti. Un opcode non implementato diventa `db` di un solo byte, così un elenco riprende dal byte successivo.

I. Caricatore, debugger e CLI

a) Caricatore: Il caricatore, nel core, è una funzione che riceve i byte del programma e non fa I/O: non legge file, modifica solo la memoria e la CPU che riceve. Prepara lo stato di un programma in stile .COM: memoria azzerata, programma copiato a 1000h:0100h, CS, DS, ES e SS uguali a 1000h, SP a FFFEh e gli altri registri a zero. I valori sono configurabili dalla riga di comando. Un programma che non entra nel segmento viene rifiutato senza modificare nulla.

b) Debugger: Il cuore del debugger è `execute(riga, stream)`, che esegue un comando e ne scrive il risultato: i test lo usano con uno stream in memoria, senza terminale. I quattordici comandi stanno in una tabella con nome, alias, sintassi, descrizione e gestore, da cui si genera anche l'help. Ci sono comandi per eseguire (`step`, `next`, `continue`), per i breakpoint (`break`, `delete`), per esaminare e modificare lo stato (`regs`, `flags`, `x`, `set`, `disasm`) e per gestire il programma (`load`, `reset`). Indirizzi e valori sono esadecimali, i conteggi decimali; un indirizzo è `seg:off` o un solo offset, e segmento e offset possono essere anche nomi di registro (`x ss:sp`).

Alcuni dettagli del comportamento:

- i breakpoint si confrontano per indirizzo fisico, quindi fermano anche quando lo stesso byte è raggiunto con un'altra coppia segmento:offset. Ripartendo da un breakpoint, la prima istruzione si esegue senza fermarsi;
- `next` esegue una CALL fino al ritorno, con un breakpoint temporaneo sull'istruzione successiva. Si ferma però solo se ci arriva con lo stesso SP di prima della CALL: in una ricorsione le chiamate più interne passano dallo stesso indirizzo con SP più basso, e senza questo controllo `next` si fermerebbe troppo presto;
- con `continue` e `next` l'esecuzione si ferma su un breakpoint, su HLT, su un opcode non implementato, su Ctrl-C o al raggiungimento di un limite di istruzioni (un milione, configurabile), che protegge dai cicli infiniti; `step` esegue invece il numero di istruzioni richiesto e ignora i breakpoint. Dopo HLT serve `reset`.

c) CLI: L'eseguibile legge i comandi dallo standard input e mostra il prompt solo se l'input è un terminale: con i comandi presi da un file l'output resta confrontabile con un risultato atteso, ed è così che viene provata l'intera CLI (Sezione VI-E). Ctrl-C richiede attenzione, perché un gestore di segnale può fare ben poco in modo sicuro: quello di SIGINT scrive soltanto un `std::atomic<bool>`, di cui un `static_assert` garantisce che sia lock-free. Il ciclo di esecuzione lo controlla prima di ogni istruzione, e lo azzera all'inizio di ogni esecuzione, così un Ctrl-C premuto al prompt non interrompe il comando successivo. La Fig. 8 mostra una parte della sessione usata come test di regressione.

```
i8086> b 120
breakpoint 1 at 1000:0120
i8086> c
breakpoint 1 at 1000:0120
*> 1000:0120  F4            hlt
i8086> x 200 16
1000:0200  00 00 01 00 01 00 02 00 03 00 05 00 08 00 0D 00  |................|
i8086> regs
AX=6FF1  BX=0000  CX=0000  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0230
DS=1000  ES=1000  SS=1000  CS=1000  IP=0120  FLAGS=F046 [ZF PF]
*> 1000:0120  F4            hlt
```

Figura 8. Parte della sessione di prova con il programma `fibonacci`, con il prompt come in un uso interattivo. L'asterisco indica un breakpoint, la freccia l'istruzione corrente.

IV. Casi limite e scelte notevoli

Alcuni casi non hanno una risposta ovvia: il manuale li descrive in modo ambiguo, la suite non li prova, oppure l'emulatore ha dovuto prendere una decisione. Questa sezione li raccoglie; la Tabella IV riassume per ognuno la fonte e il modo in cui è verificato.

A. Quando conta l'ordine delle operazioni

Il manuale descrive ogni istruzione come una sequenza di passi. Se un'istruzione legge e modifica lo stesso registro, l'ordine dei passi decide il risultato. Succede in tre casi del sottoinsieme, e in due di questi il testo preso alla lettera non coincide con l'hardware.

a) PUSH SP: PUSH decrementa SP di 2 e poi copia l'operando in cima allo stack (p. 2-31). Con SP come operando viene quindi salvato il valore già decrementato, ed è ciò che mostrano i 10.000 test della suite per l'opcode 54h.

b) POP SP: Per POP il manuale descrive l'ordine inverso: la parola in cima allo stack viene copiata nella destinazione, poi SP viene incrementato di 2 (p. 2-31). Alla lettera, con SP come destinazione il risultato sarebbe il valore letto più 2. L'hardware invece lascia in SP il valore letto, in tutti i 10.000 test dell'opcode 5Ch. L'emulatore incrementa SP prima di scrivere la destinazione.

c) CALL indiretta: Per una CALL indiretta il manuale descrive prima il salvataggio di IP sullo stack e poi la lettura della destinazione dal registro o dalla memoria (p. 2-44). L'hardware legge la destinazione *prima* del push: CALL SP salta al valore che SP aveva prima del decremento, come mostrano i 328 test di questa forma tra quelli di FF /2. Lo stesso ordine conta per una CALL che legge la destinazione da una parola di memoria sovrascritta dal push.

Dove testo e hardware divergono, il manuale descrive l'effetto dell'istruzione nel caso comune, non l'ordine interno dei passi, e decide l'hardware. Nell'emulatore non servono casi speciali: basta che ogni gestore legga e scriva nell'ordine giusto (Sezione III-G). PUSH decrementa SP e poi legge l'operando; POP legge la parola, incrementa SP e poi scrive la destinazione; CALL legge la destinazione e poi salva IP.

B. Ai bordi del segmento

Gli offset sono aritmetica modulo 64 KB: dalla fine di un segmento si torna all'inizio dello stesso segmento (p. 2-12). La regola ha conseguenze in più punti:

- una parola all'offset FFFFh ha il byte basso a *seg*:FFFFh e il byte alto a *seg*:0000h, non nel byte fisico successivo;
- un PUSH con SP = 0 scrive a SS:FFFEh, e con SP = 1 la parola sta a cavallo della fine del segmento;
- un'istruzione che inizia vicino alla fine di CS continua da CS:0000h, e la destinazione di un salto relativo è calcolata a 16 bit;
- l'indirizzo effettivo è una somma troncata a 16 bit.

La suite però non arriva a questi bordi. Abbiamo cercato i casi nei 1.340.000 test usati: nessun PUSH o CALL parte con SP uguale a 0 o 1 (il valore minimo è 2), un solo POP parte con SP = FFFFh, nessuna istruzione attraversa la fine di CS (al massimo IP più lunghezza vale FFF6h). Inoltre, nei 130.000 test di MOV e di indirizzamento controllati prima di scrivere il codice (Sezione VI-B), nessuna parola cade all'offset FFFFh. Per ciascuno di questi casi c'è quindi un test unitario dedicato, scritto a partire dalla regola del manuale.

a) Più prefissi: La suite ha 494.621 test con un prefisso di segmento e nessuno con due. La regola dell'emulatore, per cui vale l'ultimo prefisso, è quindi una scelta che la suite non può verificare. Nelle pagine consultate il manuale non fissa un numero massimo di prefissi; senza un limite, però, un segmento pieno di prefissi bloccherebbe la decodifica, e l'emulatore segnala come non implementata un'istruzione con più di otto prefissi.

C. HLT e la fine dei programmi

a) IP dopo HLT: HLT porta il processore nello stato di halt, da cui esce con un RESET o con un'interruzione; il manuale lo presenta come alternativa a un ciclo infinito quando un programma deve attendere un'interruzione (p. 2-48). Non dice però dove punti IP. Lo si può dedurre: perché HLT serva a quello scopo, dopo l'interruzione l'esecuzione deve riprendere dall'istruzione successiva; se IP restasse su HLT, al ritorno il processore si fermerebbe di nuovo. Nell'emulatore IP avanza oltre F4h come per ogni altra istruzione. La suite non ha un file per F4h: il comportamento è verificato da un test unitario e dalla sessione di prova, dove dopo l'HLT a 1000:0120 la CPU risulta ferma a 1000:0121.

b) Fine dei programmi: Il caricatore prepara uno stato in stile .COM, con SP = FFFEh e la memoria azzerata. Un RET alla fine del programma leggerebbe quindi 0000h e salterebbe a CS:0000h, dove i byte `00 00` formano `ADD [BX+SI],AL`, un'istruzione del sottoinsieme. Che cosa succede poi dipende dallo stato. Se AL non è zero, queste ADD scrivono nel codice stesso, perché CS e DS coincidono, finché compare un opcode non implementato: con il programma `mov ax,1` seguito da RET, l'emulatore si ferma su C0h a 1000:0000h. Se AL è zero, la memoria non cambia, l'esecuzione attraversa i 256 byte azzerati, riparte dal programma a 0100h e prosegue fino al limite di istruzioni. Per questo i programmi terminano sempre con HLT, e il caricatore non scrive nulla a CS:0000h. È stata scartata l'alternativa di scrivere un HLT a CS:0000h: avrebbe aggiunto al caricatore un comportamento da documentare e da provare, senza un vero vantaggio. Un RET finale per errore porta comunque a una fermata, per un opcode non implementato o per il limite di istruzioni, e il debugger mostra dove.

Tabella IV
CASI LIMITE: FONTE E VERIFICA

Caso	Fonte	Verifica
PUSH SP	manuale, p. 2-31	suite (10.000), unitario
POP SP	hardware (testo di p. 2-31 diverso)	suite (10.000), unitario
CALL SP	hardware (testo di p. 2-44 diverso)	suite (328), unitario
Parola a FFFFh	manuale, p. 2-12	unitario
Wrap dello stack	manuale, pp. 2-12, 2-14	unitario (suite: 1 POP)
Wrap di IP	manuale, p. 2-12	unitario
Più prefissi	scelta	unitario
IP dopo HLT	deduzione da p. 2-48	unitario, sessione
MOV CS	scelta (salto far)	unitario

D. MOV verso CS

Gli opcode 8Ch e 8Eh copiano un registro di segmento da o verso un operando r/m, con il registro indicato nel campo REG (Tabella 4-13). Con 8Eh e REG = 1 la destinazione sarebbe CS. Cambiare CS senza cambiare IP equivale però a un salto *far*, perché l'istruzione successiva verrebbe letta dal nuovo CS:IP; i salti far sono fuori dal sottoinsieme, quindi questa forma non è implementata. La suite non contiene test di 8Eh con REG = 1, né con REG = 5, che sull'hardware equivale a REG = 1 (Sezione V). Leggere CS con 8Ch, invece, è ammesso.

V. DOCUMENTAZIONE E HARDWARE

Il manuale e l'hardware misurato dalla suite non coincidono sempre. Nella maggior parte dei casi si tratta di codifiche che il manuale dichiara *not used*, ma che il processore esegue comunque in un modo preciso. Serviva quindi una regola per decidere quale dei due seguire.

A. La regola

La regola adottata è che il manuale è la specifica: l'emulatore riproduce l'8086 documentato, e la suite serve a verificarlo, non a definirne il comportamento. Le forme che il manuale dichiara non usate restano quindi non implementate, anche quando l'hardware dà loro un significato. Le eccezioni sono due:

- le forme 83h con REG = 1, 4, 6 (OR, AND e XOR con un immediato esteso con segno), che il manuale dichiara non usate ma che il sottoinsieme richiede esplicitamente;
- i casi in cui il manuale non dice che cosa succede: il valore dei bit riservati di FLAGS, che dichiara indeterminato (Sezione II-C), e l'ordine interno dei passi quando cambia

il risultato, come in POP SP e nella CALL indiretta (Sezione IV-A). Qui decide l'hardware.

La Tabella V riassume tutti i casi. Per le forme non usate, la colonna dell'hardware riporta lo stato assegnato dalla suite e il risultato di uno script che ha confrontato tutti i test di quella forma con una regola candidata; per 83h, che è implementata, la verifica è l'harness stesso.

B. Due osservazioni

a) L'opcode 82h: L'opcode 82h è un caso curioso, perché le tre fonti lo vedono in tre modi diversi. Il manuale lo documenta come 80h, con un immediato a 8 bit, per ADD, ADC, SBB, SUB e CMP (Tabella 4-13); la suite lo classifica come `alias` [2]; ndisasm non lo decodifica affatto e scrive `db 0x82`. L'emulatore ne implementa le forme documentate e presenti nel sottoinsieme (ADD, SUB e CMP), con una propria sotto-tabella. Per il disassemblatore, però, su 82h manca un oracolo.

b) Il costo della scelta: Seguire l'hardware non sarebbe stato più difficile: l'alias di 8Ch/8Eh costerebbe una sola riga, la maschera `REG & 3`, e per C6h/C7h basterebbe ignorare il campo REG. La scelta dipende quindi solo dalla regola, non dal costo dell'implementazione.

C. Conseguenze sui risultati

Le forme non implementate per scelta non sono errori. L'harness conta come *saltati* i loro test, separati da quelli falliti: sono 27.539, cioè i 10.039 di 8Ch/8Eh e i 17.500 di C6h/C7h, che la suite raccoglie nello stesso file delle forme implementate. Le forme 82h con REG = 1, 4, 6, invece, hanno file propri, che non vengono nemmeno scaricati: l'elenco dei file lo ricava il decoder (Sezione VI-C).

VI. Metodo di verifica e test

Un emulatore si può dire corretto solo rispetto a qualcosa. Questa sezione descrive i riferimenti usati, come sono stati usati durante lo sviluppo e quali test ne sono nati.

A. Tre oracoli

Gli oracoli, cioè le fonti del comportamento atteso, sono tre (Tabella VI): il manuale Intel [1] per la semantica documentata, la suite SingleStepTests [2] per il comportamento dell'hardware reale e ndisasm, il disassemblatore di NASM [5], per il testo prodotto dal disassemblatore. Nessuno dei tre basta da solo: ognuno copre i limiti degli altri.

B. Prima la regola, poi il codice

Il progetto è stato sviluppato in sei milestone: (1) memoria, registri, flag, decoder e ALU; (2) ModR/M, indirizzo effettivo, segmenti e prefissi; (3) salti, stack, CALL e RET; (4) disassemblatore, debugger e CLI; (5) harness di SingleStepTests; (6) programmi di prova e documentazione. Per ognuna delle prime tre, le regole da implementare, cioè le formule dei flag, il calcolo dell'indirizzo effettivo, il segmento predefinito, l'ordine dei passi di stack e salti, sono state prima scritte in un piccolo script Python usa-e-getta e confrontate con tutti i test della suite che le riguardano. Solo dopo è stato scritto il codice C++ (Tabella VII).

Il vantaggio è che un errore di comprensione emerge subito, come una regola che non torna su pochi test ben riconoscibili, invece che più tardi come un test fallito dell'emulatore, dove la causa può stare nel decoder, nell'esecutore o nella regola stessa. Gli script hanno anche mostrato proprietà della suite che hanno guidato le scelte successive: ogni test ha al più un prefisso, nessuna parola nei file di indirizzamento cade all'offset FFFFh, 8Ch/8Eh con REG 4–7 si comportano come REG & 3 e 82h si comporta come 80h.

C. SingleStepTests e l'harness

a) La suite: La suite contiene un file per ogni opcode, oppure per ogni coppia opcode e campo REG negli opcode di gruppo, con 10.000 test ciascuno [2]. Ogni test dà i byte dell'istruzione, lo stato iniziale (registri e celle di memoria) e lo stato finale, che elenca solo i registri e le celle cambiati. Il file `metadata.json` indica per ogni forma uno stato (`normal`, `alias`, `undefined`, ...) e una maschera dei flag che l'istruzione lascia indefiniti. Il progetto usa il set 8088, versione 2, a un commit fissato: uno script scarica e decomprime i file del sottoinsieme, circa 350 MB compressi e 2,6 GiB decompressi.

b) Un test: L'harness esegue ogni test in quattro passi:

1) azzera la CPU e carica lo stato iniziale: registri, FLAGS (con i bit riservati normalizzati) e celle di memoria, scritte per indirizzo fisico;
2) esegue *una* chiamata a `step()`, prefissi compresi;
3) confronta lo stato con quello atteso: tutti i registri esattamente, FLAGS solo sui flag definiti, cioè $(nostri \oplus attesi)\ \&\ maschera$, dove la maschera esclude i bit riservati e i flag indefiniti dell'istruzione (per gli opcode di gruppo dipende dal campo REG), e ogni cella di memoria citata nel test;
4) riporta a zero le celle del test, così il test successivo trova la memoria vuota senza dover azzerare un megabyte.

I campi della suite sulla coda e sui cicli di bus si ignorano (Sezione II-H). Ogni test ha tre esiti possibili: superato, fallito o *saltato*, quando l'emulatore risponde che l'istruzione non è implementata; i test saltati sono raggruppati per opcode e campo REG. L'intera suite del sottoinsieme richiede poco più di un minuto con la build ottimizzata.

c) Il sottoinsieme lo decide il decoder: L'elenco dei file da scaricare e da eseguire non è scritto a mano: l'harness lo ricava interrogando `decode()` con ogni opcode e ogni valore del campo REG. Emulatore, harness e dati scaricati non possono quindi divergere, e aggiungere un'istruzione alla tabella degli opcode aggiunge da sola i suoi test.

d) Un limite noto: L'harness controlla solo le celle elencate nel test: una scrittura dell'emulatore in un'altra cella non verrebbe rilevata, perché servirebbe confrontare tutto il megabyte a ogni test. In pratica il rischio è basso, perché un indirizzo sbagliato fa mancare anche la scrittura attesa, e quella viene rilevata.

Tabella V
CASI IN CUI MANUALE E HARDWARE NON COINCIDONO

Forma	Manuale	Hardware (suite)	Emulatore
8Ch/8Eh, REG = 4–7	*not used* (Tabella 4-13, p. 4-31)	come REG & 3: 10.039 test coerenti; stato `normal` (per l'intero opcode)	non implementata; test saltati
82h, REG = 1, 4, 6	*not used* (Tabella 4-13, p. 4-31)	come 80h (OR, AND, XOR): 7.499 test con operando registro coerenti (la parte di REG = 1, 4, 6 dei 15.115 della Tabella VII); stato `alias`	non implementata
83h, REG = 1, 4, 6	*not used* (Tabella 4-13, p. 4-31)	OR, AND, XOR con immediato esteso; stato `normal`; 30.000 test superati dall'emulatore	implementata (richiesta dal sottoinsieme)
C6h/C7h, REG = 1–7	*not used* (Tabella 4-13, p. 4-33)	campo REG ignorato, come MOV: 17.500 test coerenti; stato `undefined`	non implementata; test saltati
Bit riservati di FLAGS	valore indeterminato (Fig. 2-32)	sempre F002h	come l'hardware
POP SP, CALL indiretta	ordine dei passi nel testo (pp. 2-31, 2-44)	ordine diverso (Sezione IV-A)	come l'hardware

Tabella VI
GLI ORACOLI E I LORO LIMITI

Oracolo	Che cosa verifica	Limiti
Manuale Intel	semantica documentata	descrive l'effetto, non sempre l'ordine dei passi; non tratta il wrap oltre FFFFFh né le forme non usate
SingleStepTests	stato dopo una istruzione, misurato sull'hardware	registrata su un 8088; al più un prefisso per test; non raggiunge i bordi del segmento; nessun test di HLT
ndisasm	testo del disassemblatore	non verifica la semantica; non decodifica 82h

Tabella VII
REGOLE VERIFICATE SULLA SUITE PRIMA DI SCRIVERE IL CODICE

Milestone	Regole	Test
1	formule dei flag delle operazioni ALU, INC, DEC, NEG	149.951
2	indirizzo effettivo, segmento predefinito, prefissi, lunghezza delle istruzioni, MOV con i segmenti	130.000
2	82h si comporta come 80h (test con operando registro, REG = 0, 1, 4–7)	15.115
3	stack, PUSH e POP SP, salti, CALL e RET	160.000
	Totale	455.066

D. Controllare che i controlli sappiano fallire

Un controllo che dà sempre esito positivo non dimostra nulla. Per questo gli strumenti principali di verifica sono stati provati anche su dati sbagliati di proposito (gli script usa-e-getta delle milestone, invece, no):

- uno script altera quattro test della suite: un registro atteso, una cella di memoria attesa e un flag definito devono dare un fallimento, con la differenza esatta, mentre un bit riservato di FLAGS deve essere ignorato. Su dieci test l'harness dà, come previsto, tre falliti e sette superati;
- il confronto del disassemblatore con ndisasm è stato eseguito con un ndisasm finto, che altera la prima o l'ultima istruzione: il test fallisce e mostra la differenza. Questa prova ha trovato un errore nello script di confronto stesso. Lo script filtra l'uscita di ndisasm con `awk`, che usciva appena superato il codice; con l'opzione `pipefail` della shell, il programma che scriveva ancora nella pipe riceveva SIGPIPE e lo script terminava con il codice 141, senza mostrare alcuna differenza. Il controllo falliva, ma per il motivo sbagliato. Ora `awk` legge tutto l'input;
- anche gli script scritti per questo resoconto hanno un controllo: quello su C6h/C7h (Sezione V), rieseguito con un immediato sbagliato, fallisce su 17.481 test su 17.500 (i 19 che passano sono parole C7h con i due byte dell'immediato uguali: per la memoria lo script controlla che ogni cella scritta contenga un byte dell'immediato, non quale, e il byte alterato resta tra quelli ammessi), e quello sui bordi del segmento (Sezione IV-B) riporta anche i valori estremi trovati, per mostrare che i suoi criteri sanno trovare qualcosa.

E. Test unitari e di integrazione

CTest esegue 127 test. 123 sono casi di test Catch2 su memoria, registri, flag, ALU, decoder, CPU, disassemblatore, caricatore, analisi dei comandi e debugger, più i programmi di prova eseguiti fino a HLT. Uno esegue la CLI su un file di comandi e ne confronta l'output con quello atteso (la sessione della Fig. 8). Gli ultimi tre confrontano il disassemblatore con ndisasm sul codice dei programmi di prova, e ci sono solo se ndisasm è installato. Nella build di debug tutti i test girano con AddressSanitizer e UndefinedBehaviorSanitizer.

I test unitari hanno anche un compito che la suite non può svolgere: coprono i casi ai bordi del segmento e quelli senza test nella suite, come HLT e i prefissi multipli (Sezione IV).

F. Programmi di prova

Tre programmi in assembly NASM esercitano l'emulatore su codice realistico (Tabella VIII). Usano solo il sottoinsieme, con la direttiva `cpu 8086` che fa rifiutare a NASM le istruzioni dei processori successivi, terminano con HLT e mettono i dati da 0200h in poi. L'indirizzo fisso dei dati si ottiene riempiendo di zeri la zona di codice con la direttiva `times`: se il codice superasse 256 byte, NASM darebbe un errore invece di spostare i dati in silenzio. Un test per programma lo

Tabella VIII
I PROGRAMMI DI PROVA

Programma	Che cosa fa	Che cosa esercita
`fibonacci`	F(0)..F(23) in un array di parole	ciclo con DEC e JNZ, spiazzamento negativo, C7h
`sort`	ordinamento a bolle di dieci parole	cicli annidati, CALL e RET, PUSH e POP, scambio in memoria
`strings`	lunghezza, copia in maiuscolo e copia rovesciata di una stringa	tre subroutine, A3h, MOV con i segmenti, prefisso ES

```
mov dx, bx
sub dx, ax              ; b - a
and dx, 0x8000          ; bit di segno: 1 se b < a
jz .done
```

Figura 9. Il confronto $b < a$ con il solo JZ (da `sort.asm`): con a in AX e b in BX, il bit di segno di $b - a$ vale 1 quando $b < a$, e il salto avviene quando $b \geq a$.

Tabella IX
RISULTATI

SingleStepTests (8088, versione 2)	
File del sottoinsieme	135, di cui 134 con test (F4h non ne ha)
Test eseguiti	1.340.000
Superati	1.312.461
Falliti	**0**
Saltati per scelta	27.539 (Sezione V-C)
Durata	poco più di un minuto (Core i5-8600)
Altre verifiche	
Regole verificate prima del codice	455.066 test della suite
Test di CTest	127, tutti superati
Disassemblatore e ndisasm	52 forme campione e 87 istruzioni dei programmi, identiche
Dimensione del codice (righe totali dei file)	
Core	1.547
Debugger e CLI	844 e 119
Harness	460
Test unitari	2.167

esegue fino a HLT e controlla i dati prodotti; per `fibonacci` e `strings` controlla anche che non ci siano scritture oltre i buffer, per `sort` che SP torni a FFFEh dopo tutte le CALL e i PUSH.

a) Confronti con soli JZ e JNZ: Il sottoinsieme non ha i salti condizionati per *minore* e *maggiore*. Il confronto $b < a$ si ricava dal bit di segno della differenza $b - a$ (Fig. 9). Il metodo è corretto finché la differenza non va in overflow, cioè con valori tra 0 e 7FFFh: è il vincolo sui dati di `sort`, mentre i caratteri di `strings` lo rispettano sempre.

b) Le codifiche scelte da NASM: NASM sceglie da solo la codifica più corta, e questo ha influito sul progetto in due punti. Un'istruzione come `mov [var], ax` diventa A3h, senza ModR/M: per questo le forme A0h–A3h sono entrate nel sottoinsieme. Un immediato tra -128 e 127 su un registro a 16 bit diventa invece la forma 83h, che per AND, OR e XOR il manuale dichiara non usata: l'emulatore la implementa perché il sottoinsieme la richiede, ma i programmi la evitano, perché devono usare solo codifiche documentate. Lo script che assembla i programmi cerca con ndisasm queste forme con un registro come destinazione (non con un operando in memoria) e indica come evitarle con `strict word`; nei programmi attuali non è stato necessario. I binari assemblati sono nel repository, e riassemblandoli non devono cambiare.

G. Errori trovati dalle verifiche

La suite non ha trovato errori dell'emulatore: i suoi test sono passati tutti appena scritto il codice, un risultato che attribuiamo alle regole verificate in anticipo. Gli errori sono emersi altrove:

- il confronto con ndisasm su 52 forme campione ha trovato due errori di formato nel disassemblatore: la parola `word` superflua nelle CALL e JMP indirette in memoria, e uno spiazzamento a 16 bit dopo un registro scritto senza segno (con FFFEh ndisasm scrive `[bx-0x2]`). Dopo le correzioni le 52 forme coincidono, e coincidono anche le 87 istruzioni dei programmi di prova;
- la prova con il ndisasm finto ha trovato l'errore dello script di confronto descritto nella Sezione VI-D;
- il manuale ha corretto una convinzione iniziale: l'opcode 82h, che la suite chiama `alias`, è invece documentato (Sezione V-B);
- durante la stesura di questo resoconto, la rilettura del manuale ha mostrato che il testo su POP e CALL descrive i passi in un ordine diverso da quello dell'hardware (Sezione IV-A), e un'affermazione sui C6h/C7h ripresa dal README della suite è stata verificata con uno script.

VII. RISULTATI

La Tabella IX riassume i risultati, misurati sulla versione finale con la build ottimizzata.

Nessun test della suite fallisce. Questo significa che, per ogni forma documentata del sottoinsieme, lo stato dopo un'istruzione coincide con quello dell'hardware reale in tutti i casi casuali della suite: registri, flag definiti e memoria. I casi sono 10.000 per quasi tutte le forme, e da 1.210 a 5.027 per quelle che dividono il file con forme non implementate (C6h/C7h con REG = 0, 8Ch e 8Eh). Il risultato non copre però tutto. La suite non raggiunge i bordi del segmento e non ha test con più prefissi né di HLT: questi casi sono affidati ai test unitari, scritti a partire dal manuale (Sezioni IV e VI-E). Le scritture fuori dalle celle elencate in un test, invece, restano un limite dell'harness (Sezione VIII). I test saltati, infine, non sono errori: sono le forme che il manuale dichiara non usate e che l'emulatore, per scelta, non implementa.

VIII. LIMITI E SVILUPPI FUTURI

a) Limiti: I limiti principali sono di tre tipi:

- *il sottoinsieme*: interruzioni, I/O, istruzioni stringa, moltiplicazione e divisione, scorrimenti e rotazioni, forme far

e salti condizionati diversi da JZ e JNZ sono fuori dal progetto. Di conseguenza i programmi devono terminare con HLT;
- *il modello*: l'emulatore non ha la coda di prefetch e non conta i cicli, e per le forme che il manuale dichiara non usate non riproduce il comportamento dell'hardware;
- *la verifica*: la suite è registrata su un 8088, e i casi che non copre sono verificati solo rispetto al manuale (Sezione VII); l'harness non vede le scritture fuori dalle celle dei test; per l'opcode 82h il disassemblatore non ha un oracolo.

b) Sviluppi: Alcune estensioni sono già preparate dalla struttura del codice:

- *ADC e SBB*: l'enum delle operazioni ALU segue già la numerazione dell'8086, la tabella degli opcode li genererebbe con lo stesso ciclo, e la suite ha i loro file di test;
- *i salti condizionati 70h–7Fh*: una sola funzione di condizione, indicizzata dai quattro bit bassi dell'opcode, basterebbe per tutti e sedici;
- *coda di prefetch e cicli*: modellarli permetterebbe di confrontare anche i campi della suite che oggi l'harness ignora;
- *un'interfaccia a pannelli* nel terminale, con registri, codice disassemblato, memoria e stack visibili insieme, prevista come estensione facoltativa del progetto.

IX. Conclusioni

Il progetto ha prodotto un emulatore di un sottoinsieme dell'8086 che, su tutte le forme documentate del sottoinsieme, si comporta come l'hardware reale misurato da SingleStepTests, insieme a un debugger da terminale progettato per essere provato senza terminale.

Il lavoro ha mostrato che il manuale, pur essendo la specifica, da solo non basta: non dice nulla delle forme che dichiara non usate, e in qualche punto il suo testo, preso alla lettera, non coincide con l'hardware. Serve l'hardware misurato, e dove anche questo non arriva, come ai bordi del segmento, servono test scritti a mano a partire dalla regola documentata.

Due abitudini hanno pesato più delle altre. La prima è verificare sulla suite ogni regola che questa raggiunge, prima di scriverne il codice: gli errori di comprensione sono emersi come regole sbagliate, e i test della suite sono passati tutti appena scritto l'emulatore. La seconda è provare che i controlli principali sappiano fallire: una di queste prove ha trovato un errore proprio nello strumento di verifica.

Riferimenti bibliografici

[1] *The 8086 Family User's Manual*, Intel Corporation, 1979. [Online]. Available: https://edge.edx.org/c4x/BITSPilani/EEE231/asset/8086_family_Users_Manual_1_.pdf

[2] SingleStepTests, "8088 CPU tests, version 2," GitHub repository, commit aea84484abc79d09639d855b7b0ab32bc9e4dbeb. [Online]. Available: https://github.com/SingleStepTests/8088

[3] Catch2 contributors, "Catch2, version 3.16.1," GitHub repository. [Online]. Available: https://github.com/catchorg/Catch2

[4] N. Lohmann, "JSON for modern C++, version 3.12.0," GitHub repository. [Online]. Available: https://github.com/nlohmann/json

[5] The NASM development team, "NASM – the netwide assembler, version 2.16.01." [Online]. Available: https://www.nasm.us